

Lab no 6

Date: 18/02/2026

Programs on arrays in CUDA

Name: Satyam Singh

Reg no: 230905256

Roll No: 37

CSE B2

Lab exercise:

Q1 Write a program in CUDA which performs convolution operation on one-dimensional input array of size width using a mask array M of size mask_width to produce the resultant one-dimensional array P of size width.

Ans) #include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <cuda_runtime.h>

```
__global__ void convolution_1D(float *N, float *M, float *P,  
int w, int mw)  
{  
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (i < w) {  
    float sum = 0.0;  
    int half_mask = mw / 2;  
  
    for (int j = 0; j < mw; j++) {  
        int index = i - half_mask + j;  
        if (index >= 0 && index < w) {  
            sum += N[index] * M[j];  
        }  
    }  
    P[i] = sum;  
}  
  
int main() {  
    int N, M;  
    printf("Enter the value of width: \n");  
    scanf("%d", &N);  
    printf("Enter the value of mask_width: \n");  
    scanf("%d", &M);  
    float *h_N, *h_M, *h_P;  
    float *d_N, *d_M, *d_P;
```

```
int sizeN = N * sizeof(float);
int sizeM = M * sizeof(float);
h_N = (float*)malloc(sizeN);
h_M = (float*)malloc(sizeM);
h_P = (float*)malloc(sizeN);
printf("Enter the values for array N :\n");
for(int i = 0; i < N; i++){
    scanf("%f", &h_N[i]);
}
printf("Enter the values for mask array M :\n");
for(int i = 0; i < M; i++){
    scanf("%f", &h_M[i]);
}
cudaMalloc((void**)&d_N, sizeN);
cudaMalloc((void**)&d_M, sizeM);
cudaMalloc((void**)&d_P, sizeN);
cudaMemcpy(d_N, h_N, sizeN,
cudaMemcpyHostToDevice);
cudaMemcpy(d_M, h_M, sizeM,
cudaMemcpyHostToDevice);
dim3 dimblock(256, 1, 1);
```

```
dim3 dimgrid(ceil(N / 256.0), 1, 1);
convolution_1D<<<dimgrid, dimblock>>>(d_N, d_M, d_P,
N, M);

cudaMemcpy(h_P, d_P, sizeN,
cudaMemcpyDeviceToHost);

printf("Resultant Array P:\n");
for (int i = 0; i < N; i++) {
printf("%f ", h_P[i]);
}
printf("\n");
cudaFree(d_N);
cudaFree(d_M);
cudaFree(d_P);
free(h_N);
free(h_M);
free(h_P);
return 0;
}
```

Output:

```

STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ nvcc q1.cu -o q1
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ ./q1
Enter the value of width:
8
Enter the value of mask_width:
5
Enter the values for array N :
0 1 2 3 4 5 6 7
Enter the values for mask array M :
5 4 3 4 2
Resultant Array P:
8.000000 17.000000 30.000000 48.000000 66.000000 84.000000 86.000000 70.000000
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$

```

Q2) Write a program in CUDA to perform selection sort in parallel.

Ans)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <cuda_runtime.h>
```

```
__global__ void selesort(int *A, int *O, int n)
```

```
{
```

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
```

```
if (tid < n)
```

```
{
```

```
int pos = 0;
```

```
int data = A[tid];
```

```
for (int i = 0; i < n; i++)
```

```
{
if (A[i] < data || (A[i] == data && i < tid))
{
pos++;
}
}
O[pos] = data;
}
}
int main()
{
int N;
printf("Enter the value of size:\n");
scanf("%d", &N);
int *h_N, *h_M;
int *d_N, *d_M;
int sizeN = N * sizeof(int);

h_N = (int *)malloc(sizeN);
h_M = (int *)malloc(sizeN);
printf("Enter the values for array N:\n");
```

```
for (int i = 0; i < N; i++)
{
    scanf("%d", &h_N[i]);
}

cudaMalloc((void **)&d_N, sizeN);
cudaMalloc((void **)&d_M, sizeN);
cudaMemcpy(d_N, h_N, sizeN,
    cudaMemcpyHostToDevice);

dim3 dimblock(256, 1, 1);
dim3 dimgrid((int)ceil((float)N / 256), 1, 1);
selesort<<<dimgrid, dimblock>>>(d_N, d_M, N);

cudaMemcpy(h_M, d_M, sizeN,
    cudaMemcpyDeviceToHost);

printf("Sorted Array:\n");
for (int i = 0; i < N; i++)
{
    printf("%d ", h_M[i]);
}

printf("\n");

cudaFree(d_N);
cudaFree(d_M);
```

```
free(h_N);  
free(h_M);  
return 0;  
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ nvcc q2.cu -o q2  
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ ./q2  
Enter the value of size:  
5  
Enter the values for array N:  
6 1 9 4 2  
Sorted Array:  
1 2 4 6 9
```


Q3) Write a program in CUDA to perform odd even transposition sort in parallel.

Ans)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include "cuda_runtime.h"
```

```
#include "device_launch_parameters.h"
```

```
__global__ void oddEven(int *arr, int n) {
```

```
int tid = blockIdx.x*blockDim.x + threadIdx.x;
```

```
if (tid%2 != 0 && tid < n-1)
```

```
if (arr[tid] > arr[tid+1]){
```

```
int t = arr[tid];  
arr[tid] = arr[tid+1];  
arr[tid+1] = t;  
}  
}
```

```
__global__ void evenOdd(int *arr, int n) {  
    int tid = blockIdx.x*blockDim.x + threadIdx.x;  
    if (tid%2 == 0 && tid < n-1)  
    if (arr[tid] > arr[tid+1]){  
        int t = arr[tid];  
        arr[tid] = arr[tid+1];  
        arr[tid+1] = t;  
    }  
}
```

```
int main(void){  
    int n;  
    int *d_a;  
  
    printf("Enter length of Array : ");
```

```
scanf("%d", &n);
```

```
int size = sizeof(int)*n;
```

```
int *a = (int*)malloc(size);
```

```
cudaMalloc((void **)&d_a, size);
```

```
printf("Elements of Array : \n");
```

```
for (int i=0; i<n; i++)
```

```
scanf("%d", &a[i]);
```

```
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
```

```
dim3 dimGrid(ceil(n/256.0), 1, 1);
```

```
dim3 dimBlock(256, 1, 1);
```

```
for (int i=0; i<ceil(n/2.0); i++){
```

```
oddEven<<<dimGrid, dimBlock>>>(d_a, n);
```

```
evenOdd<<<dimGrid, dimBlock>>>(d_a, n);
```

```
}
```

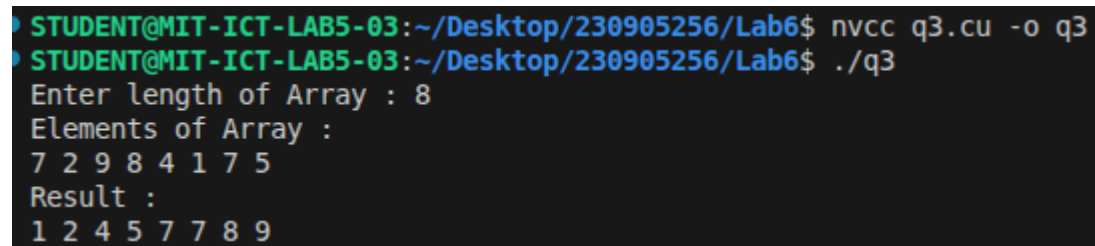
```
cudaMemcpy(a, d_a, size, cudaMemcpyDeviceToHost);
```

```
printf("Result : \n");
```

```
for (int i=0; i<n; i++)
```

```
printf("%d ", a[i]);  
printf("\n");  
cudaFree(d_a);  
return 0;  
}
```

Output:



```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ nvcc q3.cu -o q3  
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab6$ ./q3  
Enter length of Array : 8  
Elements of Array :  
7 2 9 8 4 1 7 5  
Result :  
1 2 4 5 7 7 8 9
```